Why traditional supply-chain optimization software fails under dynamic volatility

Supply-chain optimization software can help teams evaluate plans against defined objectives and constraints. The difficulty is not that optimization is inherently unsuitable. It is that real operating conditions can change after a model is run: supplier capacity shifts, a shipment is delayed, a quality hold appears or a customer commitment changes. Teams need a governed way to decide whether to accept, adapt or override a recommendation.

An optimized plan depends on its assumptions

Every model has a scope, data set, objective and set of constraints. A plan may be mathematically valid for the inputs provided and still be unsuitable if the data is stale, an operational rule is missing or the business priority has changed. Volatility exposes the gap between model output and the decision that an authorised person must make now.

That does not mean deterministic optimisation should be discarded. It means teams should make assumptions visible, monitor changes that invalidate them and connect the recommendation to a decision process. AI can help interpret new information or compare scenarios, but it does not remove the need to check constraints and authority.

Decide when to accept or override

Define the conditions under which a recommendation may proceed, the evidence required for approval and the threshold that triggers escalation. For a logistics plan, this might include service impact, cost, carrier availability and contractual constraints. For production planning, the relevant limits may include materials, maintenance, labour, quality and downstream commitments.

When an operator overrides a recommendation, record the reason and outcome. Was the model missing a fact? Did the policy change? Was a local constraint not represented? Did the execution system reject the plan? This feedback helps distinguish model limitations from poor data or a governance gap.

Connect planning to execution carefully

Approval should apply to a specific action and scope. If the route, quantity, schedule or cost changes after approval, the workflow should check whether another approval is required. The execution system should return a response, and the outcome should be reconciled with the approved decision. Keep a manual fallback for cases the model or integration cannot handle.

Where Aidecis fits

Aidecis keeps one authority model and one decision record across your people, your own algorithms and every vendor’s agents. Aidecis never writes into your systems: your people, systems or bots execute the approved action, and Aidecis reads each execution back and reconciles it against the approval, so SAP stays your single source of truth. Where you want enforced control, your own integration checks Aidecis's single-use grant before it executes. Agents and algorithms that write in their own systems are checked against their registered role and limits afterwards, and flagged within minutes if they went outside them: observed control, which does not prevent an action but shows it. An optimiser keeps its logic and is governed as one more actor; Aidecis works alongside optimisation tools rather than replacing them. The workflow, evidence and system connections must be validated for each use case; a pilot should test when the organisation accepts, pauses or overrides one defined recommendation.

See Aidecis’s decision lifecycle, use cases and benefits calculator.

Copyright 2026 Aidecis